

Similarity Search

- Faiss(Facebook AI Similarity Search)
 - 고차원 벡터의 유사도 검색(Similarity Search) 및 클러스터링을 위한 오픈소스 라이브러리
 - 대규모 벡터에서 빠르게 유사한 데이터를 찾는데 사용
 - 주요 특징
 - 근사 최근접 이웃(ANN, Approximate Nearest Neighbor) 탐색을 위한 다양한 알고리즘 지원
 - GPU 가속: CUDA를 활용하여 수십억 개의 연산을 병렬로 처리합니다.
 - 압축 (Quantization): 벡터의 용량을 1/10 이하로 줄여 메모리 효율을 극대화
 - 적용 분야
 - 대규모의 이미지를 검색하거나 실시간으로 엄청난 양의 특징점을 처리해야 하는 경우
 - 추천 시스템(예)수천만 명의 사용자 취향 벡터 중 나와 비슷한 유저 찾기)
 - 영상이나 문장을 벡터로 변환하여 내용 기반 검색 수행

Similarity Search

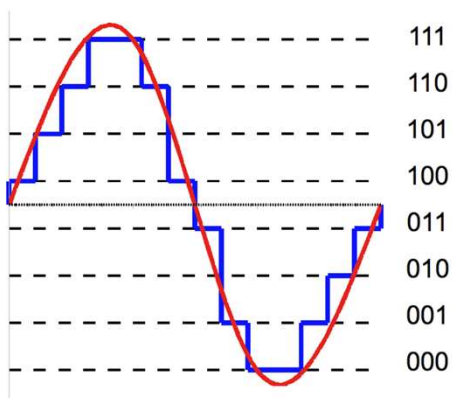
- Faiss(Facebook AI Similarity Search)
 - Faiss의 주요 알고리즘

CCDF

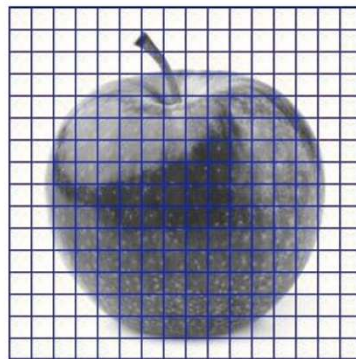
인덱스 명칭	특징	추천 상황
IndexFlatL2	전수 조사 (Exact Search). 가장 정확함	데이터가 적고(100만 개 이하) 정확도가 중요할 때
IndexIVFFlat	데이터를 여러 구역(Cell)으로 나누어 검색 범위를 제한	속도와 정확도의 균형이 필요할 때
IndexPQ	Product Quantization. 데이터를 압축하여 메모리 절약	대규모의 데이터를 메모리에 올려야 할 때
IndexHNSW	그래프 기반 탐색. 현존하는 ANN 중 가장 빠르고 정확함	메모리 여유가 있고 초고속 검색이 필요할 때

Similarity Search

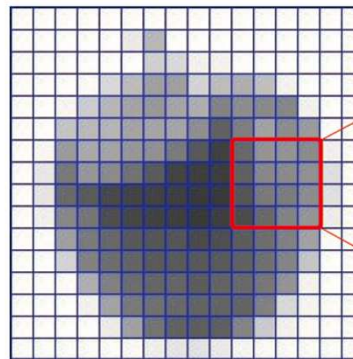
- Faiss(Facebook AI Similarity Search)
 - Product quantization
 - 고차원 벡터를 여러 부분으로 나눠 각각 양자화(quantization)하여 압축하는 방법
 - 양자화(quantization)
 - 연속적이거나 고차원의 값을 대표값으로 근사하는 것
 - Scalar quantization(스칼라 양자화)



양자화(3 bits)

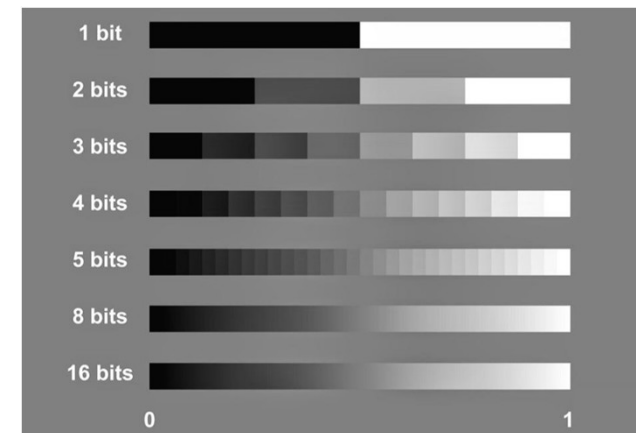


샘플링



양자화(8 bits)

105	149	149	141
97	137	139	146
86	123	126	142
76	106	132	150



Similarity Search

- Faiss(Facebook AI Similarity Search)
 - Product quantization
 - 고차원 벡터를 여러 부분으로 나눠 각각 양자화(quantization)하여 압축하는 방법
 - 양자화(quantization)
 - Vector quantization(벡터 양자화)
 1. 코드북(Codebook) 생성: 데이터 공간을 여러 구역으로 나누고 각 구역의 중심점(Centroid)을 정함. 중심점들의 모음이 코드북이 됨
 2. 인코딩(Encoding): 입력 벡터가 들어오면, 코드북에서 가장 가까운 중심점을 찾음
 3. 치환: 원래의 복잡한 벡터 데이터를 보내는 대신 그 중심점의 인덱스만 저장하거나 전송



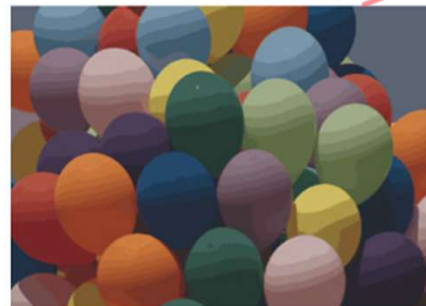
원본 영상



$k = 4$



$k = 16$



$k = 64$



$k = 256$

영상의 vector quantization 적용 예시

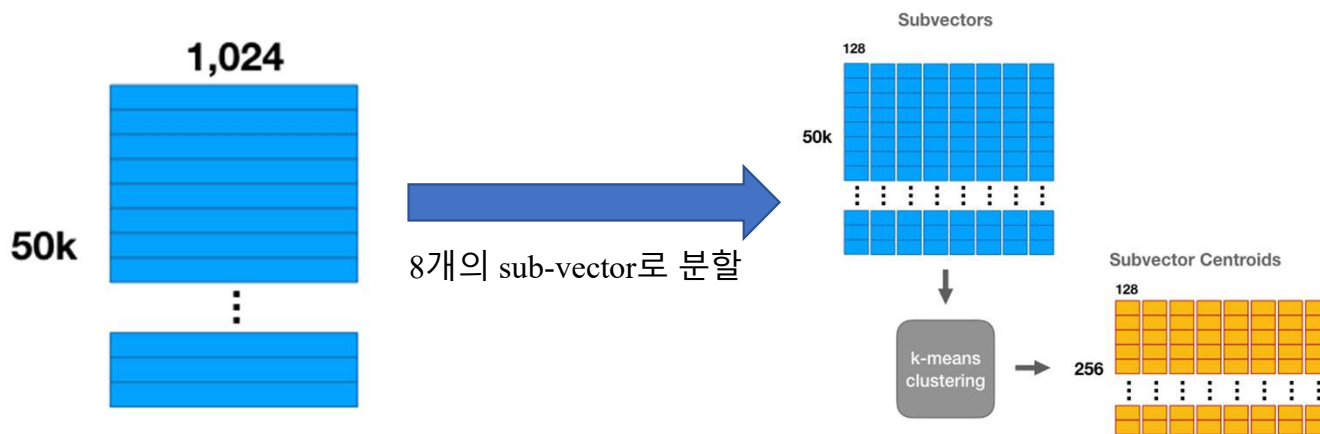
Similarity Search

- Faiss(Facebook AI Similarity Search)
 - Product quantization
 - 고차원 벡터를 여러 부분으로 나눠 각각 양자화(quantization)하여 압축하는 방법
 - 양자화(quantization)
 - Product quantization
 - 아주 긴 벡터를 여러 개의 짧은 조각으로 나눈 뒤 각 조각을 미리 정해둔 코드북으로 대체하는 방법
 - 일반적으로 원래의 데이터를 수십배 이상 줄일 수 있음
 - 예) 128차원의 SIFT 특징점 10억 개 : $128 \times 4 \times 10^9 \approx 512 \text{ GB}$
 - PQ 과정
 1. 하나의 긴 벡터를 동일한 크기의 작은 조각(Sub-vector) m개로 나눔
 2. 학습 데이터를 이용해 각 조각들의 Centroid들을 K-평균(K-means) 클러스터링으로 찾아냄
 3. 실제 데이터를 저장할 때 각 조각을 가장 닮은 centroid의 인덱스로 바꿈

Similarity Search

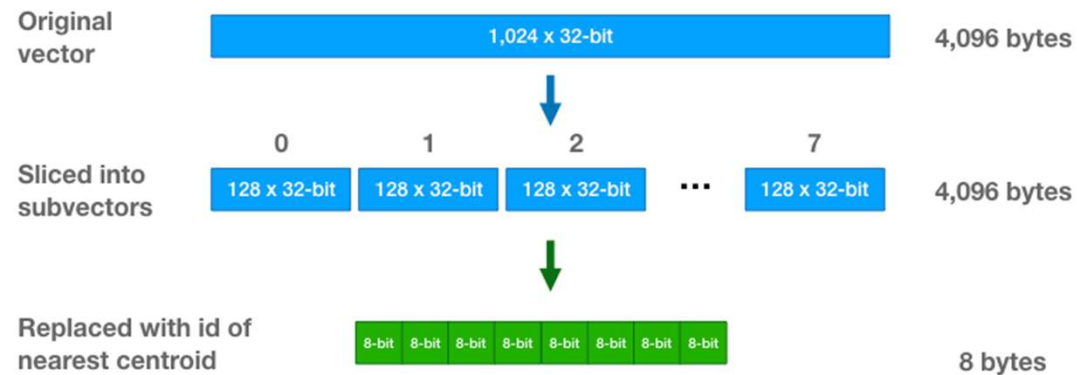
- Faiss(Facebook AI Similarity Search)
 - Product quantization
 - 고차원 벡터를 여러 부분으로 나눠 각각 양자화(quantization)하여 압축하는 방법
 - 양자화(quantization)
 - Product quantization
 - PQ 과정
 1. 하나의 긴 벡터를 동일한 크기의 작은 조각(Sub-vector) m개로 나눔
 2. 학습 데이터를 이용해 각 조각들의 Centroid들을 K-평균(K-means) 클러스터링으로 찾아냄
 3. 실제 데이터를 저장할 때 각 조각을 가장 닮은 centroid의 인덱스로 바꿈

- 50000개의 특징들
- 각 특징마다 1024차원



Similarity Search

- Faiss(Facebook AI Similarity Search)
 - Product quantization
 - 고차원 벡터를 여러 부분으로 나눠 각각 양자화(quantization)하여 압축하는 방법
 - 양자화(quantization)
 - Product quantization
 - PQ 과정
 1. 하나의 긴 벡터를 동일한 크기의 작은 조각(Sub-vector) m개로 나눔
 2. 학습 데이터를 이용해 각 조각들의 Centroid들을 K-평균(K-means) 클러스터링으로 찾아냄
 3. 실제 데이터를 저장할 때 각 조각을 가장 닮은 centroid의 인덱스로 바꿈



Similarity Search

- Faiss(Facebook AI Similarity Search)
 - Product quantization
 - 고차원 벡터를 여러 부분으로 나눠 각각 양자화(quantization)하여 압축하는 방법
 - 양자화(quantization)
 - Product quantization
 - 거리 계산
 - Query vector의 각 조각(sub-vector)과 db의 각 조각 내 클러스터들 각각의 centroid에 대해 L2 거리를 계산하여 테이블을 구축
 - 예) Query vector의 조각의 개수가 8개이고 db의 각 조각마다 256개의 centroid를 가진다고 하면 테이블의 크기는 $256 \times 8 = 2048$
 - db에 저장된 데이터와 거리를 비교 시 데이터의 각 조각에서의 인덱스에 해당하는 거리를 테이블에서 가져와서 모두 더함
 - 예) db의 데이터의 수가 50000개이고 그 중 하나의 데이터가 [12, 105, 3, 210, 45, 90, 17, 200]라고 하면
 1. 테이블의 첫번째 조각의 12번째 인덱스에 해당하는 거리값을 가져옴.
 2. 이를 여덟번째 조각까지 찾고 모두 더함

Similarity Search 실습

1. 코랩 GPU 설정

[수정] -> [노트북 설정] -> [하드웨어 가속기] -> T4 GPU로 변경

2. 설치

```
!pip install faiss-gpu-cu12
```

3. 설치 확인

```
import faiss
print(faiss.__version__)
print(faiss.get_num_gpus())
```

4. 코드 실행

```
import cv2
import numpy as np
import faiss
import time
import urllib.request
from google.colab.patches import cv2_imshow

# GPU 리소스 초기화
res = faiss.StandardGpuResources()
```

이미지 로드 및 특징점 추출 (데이터(특징점)의 수: 10,000개)

```
url1 = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/aloeL.jpg'
url2 = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/aloeR.jpg'

Urllib.request.urlretrieve(url1, 'aloeL.jpg')
Urllib.request.urlretrieve(url2, 'aloeR.jpg')

img1 = cv2.imread('aloeL.jpg', cv2.IMREAD_GRAYSCALE)
img2 = cv2.imread('aloeR.jpg', cv2.IMREAD_GRAYSCALE)

sift = cv2.SIFT_create(nfeatures=10000)
kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)

des1_f32 = des1.astype('float32')
des2_f32 = des2.astype('float32')

print(f"특징점 개수: Image1({len(kp1)}), Image2({len(kp2)})\n")
```

Similarity Search 실습

```
def run_benchmark_and_show(name, search_func, is_faiss=False):
    # --- 순수 속도 측정 (시각화 제외) ---
    search_func() # 워밍업 (GPU 메모리 할당 및 초기화 방지)

    start_time = time.time()
    for _ in range(10):
        raw_result = search_func()
    end_time = time.time()
    avg_ms = ((end_time - start_time) / 10) * 1000
    print(f"[{name}] 순수 매칭 속도: {avg_ms:.2f} ms")
    # --- [B] 시각화를 위한 후처리 (측정 범위 밖) ---
    good_matches = []
    if not is_faiss: # OpenCV Matcher (BF, FLANN)
        for m, n in raw_result:
            if m.distance < 0.7 * n.distance:
                good_matches.append(m)
    else: # Faiss (Flat, PQ)
        D, I = raw_result
        for i in range(len(desl_f32)):
            if D[i][0] < 0.7 * D[i][1]:
                m = cv2.DMatch(_queryIdx=i, _trainIdx=int(I[i][0]), _distance=D[i][0])
                good_matches.append(m)
```

```
# 매칭 결과 그리기

res_img = cv2.drawMatches(img1, kp1, img2, kp2, good_matches[:1000], None,
                           flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)

cv2.imshow(res_img)

print(f'추출된 유효 매칭 수: {len(good_matches)}\n' + "-"*80)

# OpenCV Brute-Force (CPU)

bf = cv2.BFMatcher(cv2.NORM_L2)

run_benchmark_and_show("BF-Matcher (CPU)", lambda: bf.knnMatch(des1, des2,
k=2))

# OpenCV FLANN (CPU)

flann = cv2.FlannBasedMatcher(dict(algorithm=1, trees=5), dict(checks=50))

run_benchmark_and_show("FLANN (CPU)", lambda: flann.knnMatch(des1, des2, k=2))

# Faiss FlatL2 (GPU)

index_flat = faiss.IndexFlatL2(128)

gpu_index_flat = faiss.index_cpu_to_gpu(res, 0, index_flat)

gpu_index_flat.add(des2_f32)

run_benchmark_and_show("Faiss Flat (GPU)", lambda: gpu_index_flat.search(des1_f32,
2), is_faiss=True)
```

Similarity Search 실습

```
# Faiss IVF-PQ (GPU)
# 128차원을 16개 조각(m=16)으로 나누어 8비트(nbits=8)로 양자화
nlist = 100
m = 16
quantizer = faiss.IndexFlatL2(128)
index_pq = faiss.IndexIVFPQ(quantizer, 128, nlist, m, 8)
gpu_index_pq = faiss.index_cpu_to_gpu(res, 0, index_pq)
gpu_index_pq.train(des2_f32)
gpu_index_pq.add(des2_f32)
run_benchmark_and_show("Faiss IVF-PQ (GPU)", lambda: gpu_index_pq.search(
    des1_f32, 2), is_faiss=True)
```